CSA 1737 – ARTIFICIAL INTELLIGENCE

Laboratory Record – Experiments 21 to 25

Ex. No. 21: Tic Tac Toe Game

AIM

To write a Python program to implement the Tic Tac Toe game for two players.

ALGORITHM

Step 1: Start the program.

Step 2: Create a board of 9 empty cells numbered 1 to 9.

Step 3: Player X plays first; read the position from the player and check that it is valid and empty.

Step 4: Place the player's symbol on the board and display the board.

Step 5: Check all 8 winning combinations (3 rows, 3 columns, 2 diagonals). If the current player has one, display the winner and stop.

Step 6: If the board is full and nobody has won, declare a draw.

Step 7: Otherwise switch the player and repeat steps 3 to 6.

Step 8: Stop the program.

CODE

```
# Experiment 21: Tic Tac Toe game (two players)
board = [' '] * 9

def show():
    print()
    print(f" {board[0]} | {board[1]} | {board[2]} ")
    print("---+---+---")
    print(f" {board[3]} | {board[4]} | {board[5]} ")
    print("---+---+---")
    print(f" {board[6]} | {board[7]} | {board[8]} ")
    print()

def winner(p):
    wins = [(0,1,2),(3,4,5),(6,7,8),(0,3,6),(1,4,7),(2,5,8),(0,4,8),(2,4,6)]
    return any(board[a] == board[b] == board[c] == p for a, b, c in wins)

print("TIC TAC TOE - positions are numbered 1 to 9:")
print(" 1 | 2 | 3 \n---+---+---\n 4 | 5 | 6 \n---+---+---\n 7 | 8 | 9 ")

player = 'X'
for turn in range(9):
    while True:
        pos = int(input(f"Player {player}, enter position (1-9): ")) - 1
        if 0 <= pos <= 8 and board[pos] == ' ':
            break
        print("Invalid or occupied position, try again.")
    board[pos] = player
    show()
    if winner(player):
```

```
        print("Player", player, "wins!")
        break
    player = 'O' if player == 'X' else 'X'
else:
    print("The game is a draw!")
```

OUTPUT

```
TIC TAC TOE - positions are numbered 1 to 9:
 1 | 2 | 3
---+---+---
 4 | 5 | 6
---+---+---
 7 | 8 | 9
Player X, enter position (1-9): 1

 X |   |
---+---+---
   |   |
---+---+---
   |   |

Player O, enter position (1-9): 4

 X |   |
---+---+---
 O |   |
---+---+---
   |   |

Player X, enter position (1-9): 2

 X | X |
---+---+---
 O |   |
---+---+---
   |   |

Player O, enter position (1-9): 5

 X | X |
---+---+---
 O | O |
---+---+---
   |   |

Player X, enter position (1-9): 3

 X | X | X
---+---+---
 O | O |
---+---+---
   |   |

Player X wins!
```

RESULT

Thus the Python program for the Tic Tac Toe game was written and executed successfully, and the output was verified.

Ex. No. 22: Minimax Algorithm

AIM

To write a Python program to implement the Minimax algorithm for gaming.

ALGORITHM

Step 1: Start the program.

Step 2: Represent the game tree as a binary tree whose leaf nodes contain the utility values.

Step 3: Call minimax() from the root, which is a MAX node.

Step 4: If a leaf node is reached, return its value.

Step 5: At a MAX node, return the maximum of the values of its children.

Step 6: At a MIN node, return the minimum of the values of its children.

Step 7: Display the optimal value obtained at the root.

Step 8: Stop the program.

CODE

```
# Experiment 22: Minimax algorithm for gaming
import math

def minimax(depth, node_index, is_max, scores, height):
    # Leaf node reached: return its score
    if depth == height:
        return scores[node_index]
    if is_max:
        return max(minimax(depth + 1, node_index * 2, False, scores, height),
                   minimax(depth + 1, node_index * 2 + 1, False, scores, height))
    else:
        return min(minimax(depth + 1, node_index * 2, True, scores, height),
                   minimax(depth + 1, node_index * 2 + 1, True, scores, height))

# Leaf node values of a binary game tree
scores = [3, 5, 6, 9, 1, 2, 0, -1]
height = int(math.log2(len(scores)))

print("Leaf node values :", scores)
print("Height of tree   :", height)
print("The optimal value (MAX player at root) is :", minimax(0, 0, True, scores,
height))
```

OUTPUT

```
Leaf node values : [3, 5, 6, 9, 1, 2, 0, -1]
Height of tree   : 3
The optimal value (MAX player at root) is : 5
```

RESULT

Thus the Python program to implement the Minimax algorithm was written and executed successfully; the optimal value at the root is 5.

Ex. No. 23: Alpha-Beta Pruning

AIM

To write a Python program to implement the Alpha-Beta pruning algorithm for gaming.

ALGORITHM

Step 1: Start the program.

Step 2: Represent the game tree and initialise alpha = -infinity and beta = +infinity.

Step 3: At a MAX node, update alpha with the largest value found so far.

Step 4: At a MIN node, update beta with the smallest value found so far.

Step 5: If beta <= alpha, the remaining children of that node cannot affect the result, so prune them (stop exploring).

Step 6: Return the best value to the parent node.

Step 7: Display the optimal value and the number of pruned branches.

Step 8: Stop the program.

CODE

```
# Experiment 23: Alpha-Beta Pruning algorithm for gaming
import math

pruned = 0

def alpha_beta(depth, node_index, is_max, scores, height, alpha, beta):
    global pruned
    if depth == height:
        return scores[node_index]
    if is_max:
        best = -math.inf
        for i in range(2):
            val = alpha_beta(depth + 1, node_index * 2 + i, False, scores,
height, alpha, beta)
            best = max(best, val)
            alpha = max(alpha, best)
            if beta <= alpha:
                print(f"Pruned at depth {depth} (MAX node {node_index}):
alpha={alpha}, beta={beta}")
                pruned += 1
                break
        return best
    else:
        best = math.inf
        for i in range(2):
            val = alpha_beta(depth + 1, node_index * 2 + i, True, scores, height,
alpha, beta)
            best = min(best, val)
            beta = min(beta, best)
            if beta <= alpha:
                print(f"Pruned at depth {depth} (MIN node {node_index}):
alpha={alpha}, beta={beta}")
                pruned += 1
                break
        return best
```

```
scores = [3, 5, 6, 9, 1, 2, 0, -1]
height = int(math.log2(len(scores)))

print("Leaf node values :", scores)
result = alpha_beta(0, 0, True, scores, height, -math.inf, math.inf)
print("The optimal value is :", result)
print("Number of pruned branches :", pruned)
```

OUTPUT

```
Leaf node values : [3, 5, 6, 9, 1, 2, 0, -1]
Pruned at depth 2 (MAX node 1): alpha=6, beta=5
Pruned at depth 1 (MIN node 1): alpha=5, beta=2
The optimal value is : 5
Number of pruned branches : 2
```

RESULT

Thus the Python program to implement Alpha-Beta pruning was written and executed successfully; the optimal value 5 was obtained with 2 branches pruned.

Ex. No. 24: Decision Tree

AIM

To write a Python program to implement a Decision Tree classifier.

ALGORITHM

Step 1: Start the program.

Step 2: Import the required libraries from scikit-learn and load the Iris dataset.

Step 3: Split the data into training (70%) and testing (30%) sets.

Step 4: Create a DecisionTreeClassifier with maximum depth 3 and train it using fit().

Step 5: Predict the classes of the test data and calculate the accuracy.

Step 6: Display the rules of the tree using export_text().

Step 7: Predict the class of a new sample.

Step 8: Stop the program.

CODE

```
# Experiment 24: Decision Tree (using scikit-learn on the Iris dataset)
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier, export_text
from sklearn.metrics import accuracy_score

iris = load_iris()
X, y = iris.data, iris.target

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42)

model = DecisionTreeClassifier(max_depth=3, random_state=42)
model.fit(X_train, y_train)

y_pred = model.predict(X_test)
print("Accuracy on test data :", round(accuracy_score(y_test, y_pred) * 100, 2),
"%")

print("\nDecision Tree rules:")
print(export_text(model, feature_names=list(iris.feature_names)))

sample = [[5.1, 3.5, 1.4, 0.2]]
print("Prediction for sample", sample[0], "->",
iris.target_names[model.predict(sample)[0]])
```

OUTPUT

```
Accuracy on test data : 100.0 %

Decision Tree rules:
|--- petal length (cm) <= 2.45
|   |--- class: 0
|--- petal length (cm) >  2.45
|   |--- petal length (cm) <= 4.75
|   |   |--- petal width (cm) <= 1.60
```

```
|   |   |   |--- class: 1
|   |   |--- petal width (cm) >  1.60
|   |   |   |--- class: 2
|   |--- petal length (cm) >  4.75
|   |   |--- petal width (cm) <= 1.75
|   |   |   |--- class: 1
|   |   |--- petal width (cm) >  1.75
|   |   |   |--- class: 2

Prediction for sample [5.1, 3.5, 1.4, 0.2] -> setosa
```

RESULT

Thus the Python program to implement a Decision Tree was written and executed successfully, with 100% accuracy on the test data.

Ex. No. 25: Feed Forward Neural Network

AIM

To write a Python program to implement a Feed Forward Neural Network with backpropagation, trained on the XOR problem.

ALGORITHM

Step 1: Start the program.

Step 2: Create the XOR input data and expected outputs.

Step 3: Build a network with 2 input neurons, 4 hidden neurons and 1 output neuron, and initialise the weights randomly.

Step 4: Forward pass: calculate the hidden layer and output using the sigmoid activation function.

Step 5: Calculate the error between the expected and the actual output.

Step 6: Backward pass: calculate the gradients and update the weights and biases using the learning rate.

Step 7: Repeat steps 4 to 6 for 10000 epochs, printing the loss at intervals.

Step 8: Display the predictions of the trained network.

Step 9: Stop the program.

CODE

```
# Experiment 25: Feed Forward Neural Network (with Backpropagation) - XOR problem
import numpy as np

np.random.seed(1)

# XOR training data
X = np.array([[0, 0], [0, 1], [1, 0], [1, 1]])
y = np.array([[0], [1], [1], [0]])

def sigmoid(x):
    return 1 / (1 + np.exp(-x))

def sigmoid_derivative(a):
    return a * (1 - a)

# Network: 2 input -> 4 hidden -> 1 output
W1 = np.random.uniform(-1, 1, (2, 4))
b1 = np.zeros((1, 4))
W2 = np.random.uniform(-1, 1, (4, 1))
b2 = np.zeros((1, 1))
lr = 0.5

for epoch in range(10001):
    # Forward pass
    hidden = sigmoid(np.dot(X, W1) + b1)
    output = sigmoid(np.dot(hidden, W2) + b2)

    # Error and backward pass
```

```
    error = y - output
    d_output = error * sigmoid_derivative(output)
    d_hidden = d_output.dot(W2.T) * sigmoid_derivative(hidden)

    # Update weights
    W2 += hidden.T.dot(d_output) * lr
    b2 += np.sum(d_output, axis=0, keepdims=True) * lr
    W1 += X.T.dot(d_hidden) * lr
    b1 += np.sum(d_hidden, axis=0, keepdims=True) * lr

    if epoch % 2000 == 0:
        print(f"Epoch {epoch:5d}  Loss = {np.mean(np.square(error)):.6f}")

print("\nPredictions after training:")
for inp, out in zip(X, output):
    print(inp, "->", round(float(out[0]), 4), "=>", int(out[0] > 0.5))
```

OUTPUT

```
Epoch     0  Loss = 0.249726
Epoch  2000  Loss = 0.002766
Epoch  4000  Loss = 0.000892
Epoch  6000  Loss = 0.000516
Epoch  8000  Loss = 0.000359
Epoch 10000  Loss = 0.000275

Predictions after training:
[0 0] -> 0.0145 => 0
[0 1] -> 0.9813 => 1
[1 0] -> 0.9882 => 1
[1 1] -> 0.02 => 0
```

RESULT

Thus the Python program to implement a Feed Forward Neural Network was written and executed successfully; the network learned the XOR function correctly.